Datum — Analytics Service

Service documentation · Analytics · Internal engineering documentation

1. Overview

The **analytics** service is the brain behind the numbers you see on the Datum dashboard. It's the biggest service in the platform, and it's responsible for calculating and serving almost everything merchants look at: sales KPIs, funnels, product performance, session behavior, website speed (web vitals), file uploads, API keys, and the Shopify file-upload feature. It runs on port 3006, and nothing talks to it directly — all requests come in through the api-gateway, which forwards the right traffic to it.

2. Architecture

2.1 How the Service Starts and Handles Requests

- The service starts up, connects to its databases, and sets up a clean shutdown process so that if it's restarted or stopped, it closes all its database connections properly instead of leaving them hanging.
- Before any request reaches actual business logic, it passes through a chain of checks: security headers are added, errors are tracked (via Sentry), only approved websites are allowed to call it (CORS), and incoming request size is capped at 60MB. Each feature area (metrics, dashboard, sessions, etc.) has its own set of routes, all wired together at startup.
- Internally, it listens on port 3000 by default but is addressed on the network as `analytics-service:3006`.

2.2 Reporting Its Own Health

The service tells the platform's health-monitor whenever it starts up — sharing what it depends on (MongoDB, MySQL, Redis) and what routes it exposes. While running, it also reports whenever a request fails or recovers, so the health-monitor dashboard stays up to date on whether analytics is working properly.

3. Where the Data Lives

Data store	How it's used	What's stored there
Shared MySQL database	Used for platform-wide data	API keys, saved dashboard layouts

Per-brand MySQL database	Each brand has its own database; the service looks up and connects to the right one per request	Each brand's actual sales/order/product data
Shared MongoDB (main)	General-purpose document storage	Dashboard layout customizations
MongoDB (session analytics)	Dedicated store for visitor session data	Records of user sessions — who visited, when, from where
MongoDB (web vitals)	Dedicated store for site speed data	Website performance / loading-speed measurements
Redis	Fast in-memory cache	Recently-computed metrics, so repeat requests don't have to be recalculated from scratch

Multi-brand data isolation: because Datum serves many different brands, each brand's real business data lives in its own separate MySQL database. When a request comes in, the service figures out which brand it's for, looks up that brand's database connection details from a separate "tenant-router" service, and connects to the right database for that request. This keeps every brand's data cleanly separated from every other brand's.

4. Who's Allowed to Do What

The analytics service doesn't check logins itself — it trusts that the api-gateway has already verified who the user is, and simply reads that information from the incoming request (user ID, which brand they belong to, their role, their permissions). Optionally, the gateway can cryptographically sign this information so the service can confirm it wasn't tampered with — though right now that extra check is only enforced if it's specifically turned on.

Check	What it does
Logged-in check	Confirms someone is logged in, without caring who
Author-level check	Confirms the user has an elevated role (author, admin, or super admin)
Permission check	Confirms the user has been granted a specific permission for that feature
Brand check	Figures out which brand the request is for and connects to that brand's database
API key check	Lets external tools call the API using a long-lived key instead of a user login, with its own rate limit (100 requests/minute)
Login-or-API-key check	Accepts either a normal login or an API key, whichever is present
Author-or-internal-key check	Used for admin actions — allows either an author-level user or a trusted internal system

5. What the Service Can Do (API Reference)

The tables below list every endpoint the service exposes, grouped by feature area, along with who's allowed to call each one.

5.1 Sales & Traffic Metrics — /metrics

Endpoint	What it does	Who can access it
GET /order-split	Breaks down orders by category	Logged-in users (brand-specific)
GET /payment-sales-split	Breaks down sales by payment method	Logged-in users (brand-specific)
GET /payment-split-summary	Summary of payment method usage	Logged-in users (brand-specific)
GET /payment-split-trend	Payment method usage over time	Logged-in users (brand-specific)
GET /traffic-source-split	Where visitors are coming from	Logged-in users (brand-specific)
GET /summary	The main dashboard summary numbers	Logged-in users (brand-specific)
GET /data-restriction-config	How far back in time data can be queried	Logged-in users
GET /summary/brands	Summary across every brand a user can see	Logged-in users
GET /summary-filter-options	Filter choices available on the summary screen	Logged-in users
GET /web-performance-summary	Overview of website speed	Logged-in users
GET /top-pdps	Most-visited product pages	Open to the public, or via login/API key
GET /top-products	Best-selling products	Login or API key
GET /product-kpis	Key numbers per product	Login or API key
GET /product-types	Sales broken down by product type	Login or API key
GET /hourly-trend	Hour-by-hour trend	Logged-in users (brand-specific)
GET /daily-trend	Day-by-day trend	Logged-in users (brand-specific)
GET /monthly-trend	Month-by-month trend	Logged-in users (brand-specific)
GET /daily-funnel	Conversion funnel for the day	Requires the "daily funnel" permission
GET /hourly-product-sessions/export	Download hourly product session data as a spreadsheet	Author-level users only
GET /hourly-sales-compare	Compares sales hour-to-hour	Logged-in users
GET /hourly-sales-summary	Hourly sales overview	Logged-in users
GET /diagnose/total-orders	Internal troubleshooting tool	Protected/internal use

5.2 Product Conversion — /metrics

Endpoint	What it does	Who can access it
GET /product-conversion	How well products convert browsers into buyers	Requires "product conversion" or "inventory" permission
GET /product-conversion/export	Download the above as a spreadsheet	Requires "product conversion" permission

5.3 Bundles — /metrics

Endpoint	What it does	Who can access it
GET /bundles/options	Filter choices for the bundles screen	Requires "bundles" permission
GET /bundles/summary	How product bundles are performing	Requires "bundles" permission
GET /bundles/summary/export	Download bundle summary as a spreadsheet	Requires "bundles" permission
GET /bundles/products	Breakdown of products inside bundles	Requires "bundles" permission
GET /bundles/products/export	Download bundle products as a spreadsheet	Requires "bundles" permission

5.4 Dashboard Layout — /dashboard

Endpoint	What it does	Who can access it
GET /layout	Load a user's saved dashboard layout	Requires layout-customization permission
POST /layout	Save a dashboard layout	Requires layout-customization permission

5.5 Visitor Session Analytics — /session-analytics

Every endpoint below requires session-analytics permission.

Endpoint	What it does
GET /summary	Overview of visitor sessions
GET /trend	Session activity over time
GET /brands	Session breakdown by brand
GET /brands/export	Download the brand breakdown as a spreadsheet
GET /users	Session activity by individual user
GET /users/export	Download user session data as a spreadsheet
GET /insights	Automatically surfaced session insights

GET /filters	Filter choices for session analytics

5.6 Website Speed / Web Vitals — /web-vitals

Every endpoint below requires the web-vitals permission.

Endpoint	What it does
GET /snapshot	Current site speed snapshot
GET /all-brands-snapshot	Snapshot across every brand
GET /trend	Site speed trend over time
GET /pages	Speed broken down by page

5.7 External-Facing — /external

Endpoint	What it does	Who can access it
GET /last-updated/pts	Checks how fresh the underlying data is	Logged-in users (brand-specific)

5.8 File Uploads — /

Endpoint	What it does	Who can access it
POST /upload	Uploads a file to cloud storage	**Nobody is checked — open access**
GET /uploads	Lists previously uploaded files	**Nobody is checked — open access**

5.9 API Key Management — /admin/api-keys

Endpoint	What it does	Who can access it
POST /admin/api-keys	Creates a new API key	Author-level user or internal system
GET /admin/api-keys	Lists API keys for a brand	Author-level user or internal system
POST /admin/api-keys/:id/revoke	Disables an API key	Author-level user or internal system
POST /admin/api-keys/:id/rotate	Replaces an API key with a new one	Author-level user or internal system

5.10 Shopify File Upload — /shopify

Endpoint	What it does	Who can access it
POST /upload-file	Uploads a file directly into a brand's Shopify store	Requires an API key with file-upload permission

5.11 Push Notifications — /notifications

Endpoint	What it does	Who can access it
POST /push/receive	Receives incoming push notification callbacks	**Nobody is checked — open access**
POST /subscribe	Subscribes a device to notification topics	**Nobody is checked — open access**

6. How the Numbers Actually Get Calculated

- **Dashboard summary** — the main KPI numbers merchants see are calculated by a dedicated summary service, which pulls together sales figures, filters, and timezone-adjusted date ranges.
- **Payment split & trend reporting** — a separate reporting service handles anything related to how customers paid and how that's changed over time.
- **Hourly, daily, and monthly trends, plus the daily funnel** — all handled by shared trend-calculation logic built on top of the same summary engine.
- **Product analytics** — top products, top pages, and per-product KPIs are handled by a paged, cache-aware service so large product catalogs don't slow things down.
- **Smart caching** — recently-requested numbers are kept in a short-lived cache (both in memory and in Redis) so that if the same report is requested again moments later, it doesn't have to be recalculated. It also avoids doing the same expensive calculation twice at once if two requests come in together.
- **Guardrails on huge date ranges** — asking for data across a very long time span (more than 30 days, by default) is intentionally limited, to avoid overloading the system with an enormous, slow query.
- **Consistent date handling** — a shared helper makes sure every part of the service interprets date ranges and timezones the same way.

> One file in this service, `pipeline.py`, is kept only as a reference copy. The actual system that processes and loads analytics data runs in a completely separate repository — this copy isn't live and doesn't affect production.

7. Shopify Integration

This service can push a file directly into a brand's Shopify store — for example, uploading an image or document into Shopify's file library. It does this in three steps: it asks Shopify for a temporary upload location, sends the file there, and then tells Shopify to register the file as part of the store. This is a one-way upload feature, not a two-way sync — the service doesn't receive anything back from Shopify automatically (no webhooks), and there's no login/connect flow; each brand's Shopify access is pre-configured ahead of time.

8. Something Worth Flagging

> ⚠ **Four endpoints have no access control at all**: the two file-upload endpoints (`/upload` , `/uploads`) and the two push-notification endpoints (`/push/receive` , `/subscribe`). Anyone who can reach the service can call these without logging in or providing any credentials. This is called out here so it's visible and can be prioritized — nothing has been changed as part of writing this documentation.

9. Configuration

The service is configured through environment variables, grouped below by purpose.

Category	What it controls
Basic setup	Which port to run on, which environment it's in, its service name
Error tracking & metrics	Sending errors to Sentry, exposing internal performance metrics
Allowed websites (CORS)	Which frontend domains are allowed to call this service
Shared database connection	Connection details and pool sizing for the shared MySQL database
Per-brand database connections	Pool sizing for the many per-brand MySQL connections
MongoDB connections	Connection details for the three separate MongoDB stores (main, sessions, web vitals)
Cache (Redis)	Connection details for the caching layer
Brand lookup service	How to reach the tenant-router service, how long to cache brand lookups, and the key used to decrypt stored database passwords
Trust between gateway and service	The shared secret used to confirm requests genuinely came from the gateway
Internal system access	The shared key that lets other internal services bypass normal login checks
Business thresholds	Numeric thresholds used in inventory-related alerts and calculations
Long date-range limits	How far back a single query is allowed to look before being restricted
File storage (AWS S3)	Credentials and bucket details for storing uploaded files
Shopify credentials	Store name, access token, and API version for each brand's Shopify store
Health monitoring	Where and how this service reports its health and any failures
Legacy brand config	An older, fallback way of listing brand database configs

10. Notable Tools & Libraries It Relies On

Tool	What it's used for
Express	The framework that handles incoming web requests
Sequelize & MySQL driver	Talking to the shared MySQL database
Mongoose & MongoDB driver	Talking to the MongoDB stores
DuckDB	Running fast analytical queries over large archived datasets
Redis client	The caching layer
In-memory cache library	Quick lookups without hitting the database, e.g. for brand routing
AWS S3 tools	Storing and retrieving uploaded files
Firebase Admin	Sending push notifications
HTTP request tools	Talking to Shopify's API
Password hashing	Securing stored API keys
Data validation	Checking that incoming requests are well-formed
Sentry	Tracking and reporting errors
Metrics exposition	Exposing performance stats for monitoring
Spreadsheet export	Generating the CSV downloads used throughout the service
Security headers & CORS	Basic web security protections
File upload handling	Processing uploaded files in memory before storing them
Real-time communication	Supporting live, socket-based updates
Testing tools	Automated tests that check the service works as expected